Verifier — prompts

The Verifier (Gemini 3.5 Flash) runs programmatically during Phase 2 immediately after the drafting pass ends. It executes a single, zero-turn (one-shot) API call in JSON mode, evaluating the proposed draft image against our sequential cropping rules and returning a pass/fail verification check.

System prompt

Frames the agent's role ("expert quality verifier") and embeds the specific cropping rules verbatim.

Placeholders:

- `{rules}` — the `## V.A` (Macro Quality) or `## V.B` (Micro Quality) rules section of `docs/rules.md` depending on the current pipeline step.

Template:

```
You are an expert quality verifier enforcing visual standards on
technical machine catalogs. (Agent Identity: {page_id})

CROPPING RULES FOR EVALUATION:
{rules}

GUIDELINE FOR PRACTICAL VERIFICATION (NOT OVERLY STRICT):
- BE PARTICULARLY LENIENT WITH 3D VIEWS: These represent highly complex
visual geometries. You must evaluate them with maximum practical
leniency. Do NOT reject them over minor, non-critical border clipping
or minor adjacent overlaps. If the core physical structure is
substantially visible and readable, you MUST mark it as PASS.
- Avoid pedantic or 'pixel-perfect' rejections. Minor visual
imperfections, slight boundary overlaps, or tiny sub-pixel artifacts
that do not degrade the overall legibility and professional machine-
catalog appearance MUST be accepted.
- Your default stance should be to PASS a crop unless a critical visual
failure is present (e.g. slicing directly through a BOM table or
leaving a main mechanical drawing completely severed).

Your task is to analyze the provided crop image, verify if it complies
with all cropping rules, and return a strict JSON pass/fail status.
```

Initial user message

Specifies the semantic context of the draft.

Placeholders:

- `{label}` — unique semantic label of the drafted region.
- `{desc}` — detailed description of the drafted region.

Template:

```
Analyze this proposed crop labeled '{label}' ('{desc}') and verify if
it complies with the cropping rules.
```

JSON Response Schema

Enforced natively via Google's `response_schema` parameter to guarantee 100% structured parsing reliability:

```
{
  "type": "object",
  "properties": {
    "pass_check": {
      "type": "boolean",
      "description": "Set to true if the crop perfectly complies with
every single visual cropping quality rule. Set to false if any rule
fails or is truncated."
    },
    "violations": {
      "type": "string",
      "description": "If pass is false, provide a precise, multi-
sentence critique of exactly what is wrong, clipped, truncated, or
missing from the crop. If pass is true, leave empty."
    }
  },
  "required": ["pass_check", "violations"]
}
```

Workflow Progression Summary

The programmatic loop coordinates these verification calls:

1. **Delineator finishes drafting:** All proposed crops are added to an in-memory queue.
2. **Verifier is invoked (One-Shot):** The verifier inspects the PNG.
   - *If Pass Check = True:* Python automatically moves the PNG to `crops/` and registers the region in `regions.yaml`.
   - *If Pass Check = False (Pass 1 Macro Verification only):* The Verification Controller checks the `violations` string for unrecoverable page exclusions. If a **Rule A.5 (Prose Narrative)** or **Rule A.2 (TOC / Table of Contents)** exclusion is cited:
     - **Dynamic Short-Circuit:** The controller instantly aborts the execution loop on Round 1 and discards the draft, completely bypassing any Re-Crop cycles to prevent compute and token waste.
   - *Standard Failure:* If the failure is recoverable (or during Pass 2 Micro Verification), the **Re-Crop Agent** is spawned given the verifier's critique.
3. **Iteration Limit:** The loop (Re-Crop -> Verify) runs for at most **3 rounds**. If it does not pass by Round 3, the draft is permanently discarded.